1. Preliminaries

Before starting on this assignment, please be sure to read the PostgreSQL Info file (PSQL_Info.pdf) and the Docker + PostgreSQL Setup Guide that are on Piazza under General Resources (the cse180-compose.yml file is there too). This quarter you do not install PostgreSQL on your own machine. You run psql inside the Docker container, as described in the Setup Guide.

2. Policies (read this first)

Deadline and late window. Lab1 is due by 11:59pm on Friday, October 9. Late submissions are accepted until 8:00am on Sunday, October 11 with an automatic 20% penalty. Nothing is accepted after that, for any reason. We always grade the last submission on Canvas, so a resubmission during the late window gets the penalty. Solutions are posted on Sunday, October 11.

AI policy. AI use (ChatGPT, Claude, Gemini, …) is allowed in this lab but must be disclosed: say in your README which tool you used and what for. Undisclosed AI use is an academic integrity violation. You must be able to explain, defend, and modify every line you submit, and the teaching team may quiz you on it.

Lab Check 1. In your lab section during week 3 (October 12 to 16), first 20 minutes: a short paper check on the Lab1 material, closed book, no devices, no AI. It is easy if you did and understood your own lab. No make-ups.

3. Goal

The goal of the first assignment is to create a PostgreSQL data schema with a number of different tables. That is all that is required in this assignment. The other Lab Assignments will be more challenging. In your Lab Sections, you may be given information about how to load data into a table and issue simple SQL queries, because that's fun, but loading data and issuing queries are **not** required in this assignment. (That will show up in the Lab2 assignment.)

4. Lab1 Description

4.1 Create PostgreSQL Schema Lab1

You will create a Lab1 schema to set apart the database tables created in this lab from tables you will create in future labs, as well as from tables (and other objects) in the default (public) schema. Note that the meaning of schema here is specific to PostgreSQL, and distinct from the general meaning of schema. See here for more details on PostgreSQL schemas. You create the Lab1 schema using the following command:

```
CREATE SCHEMA Lab1;
```

[PostgreSQL makes all identifiers lowercase, unless you put them in quotation marks, e.g., "Lab1". But you don't have to bother using quotation marks for identifiers. We use capitals in assignments for readability, but it's okay (and equivalent) if you use lab1 as the schema name.]

Now that you have created the schema, you want to make Lab1 the default schema when you use psql. If you do not set Lab1 as the default schema, then you will have to qualify your table names with the schema name (e.g., by writing Lab1.customer, rather than just customer). To set the default schema, you modify your search path as follows. (For more details, see here.)

```
ALTER ROLE cse180 SET SEARCH_PATH to Lab1;
```

You will need to log out and log back in to the server for this default schema change to take effect. (Type \q, then run the docker exec command from the Setup Guide again.) (Students **often forget** to do this, and then are surprised that their tables aren't in the expected schema.) To see your current SEARCH_PATH, enter:

```
SHOW SEARCH_PATH;
```

If you forget to do the ALTER ROLE for Lab1 to modify your search path then your schema will be the default schema, PUBLIC.

4.2 Tables

You will be creating a (simplified) database for a space launch company that flies payloads and astronauts on missions. This schema might not provide everything that an actual database of such a company would include, but it's a decent start.

Important: To receive full credit, you must use the attribute names as given, and the attributes must be in the order given. Do not do more than you're asked to do in this assignment.

Payload(payloadID, payloadName, owner, massKg, payloadType)

LaunchSite(siteCode, siteName, country, numLaunchPads, altitudeMeters)

Mission(missionID, launchSiteCode, scheduledLaunch, actualLaunch, rocketType, durationHours, outcome)

Astronaut(astronautID, astronautName, agency, dateOfBirth, yearsExperience)

Contract(contractID, missionID, signedDate, priority, price, paymentMethod)

PayloadIntegration(contractID, payloadID, integrationTime, numTechnicians, baySlot)

MissionCrew(missionID, astronautID, role, compensation)

The underlined attribute (or attributes) identifies the Primary Key of each table. A table can only have one Primary Key, but that Primary Key may involve multiple attributes.

- The relation **Payload** specifies the payload's ID, its name, the organization that owns it, its mass in kilograms, and its type (e.g., satellite, cargo, probe).
- The relation **LaunchSite** specifies the code of a launch site, its name and country, the number of launch pads it has, and its altitude in meters.
- The relation **Mission** specifies the ID of a mission, the code of the site it launches from, the scheduled and actual launch times, the rocket type, the duration of the mission in hours, and its outcome (e.g., success, failure).
    - Any launchSiteCode that appears in a Mission row must appear as a siteCode in the LaunchSite table.
- The relation **Astronaut** specifies the astronaut's ID, their name, the space agency they belong to, their date of birth, and their years of experience.
- The relation **Contract** records an ID for every contract to fly payloads on a mission, including the mission ID, the date the contract was signed, the priority level, the price, and the method of payment. A single contract may cover several payloads (see PayloadIntegration).
    - Any missionID that's in a Contract row must appear as a missionID in the Mission table.

- The relation **PayloadIntegration** records which payloads fly under which contracts, including the contract ID, the payload ID, the time the payload was integrated into the rocket, the number of technicians involved, and the bay slot assigned.
    - Any contractID that's in a PayloadIntegration row must appear as a contractID in the Contract table.
    - Any payloadID that's in a PayloadIntegration row must appear as a payloadID in the Payload table.
- The relation **MissionCrew** records which astronauts are assigned to which missions, including the mission ID, the astronaut ID, the role on that mission (e.g., Commander, Pilot, Specialist), and the compensation amount for that assignment.
    - Any missionID that's in a MissionCrew row must appear as a missionID in the Mission table.
    - Any astronautID that's in a MissionCrew row must appear as an astronautID in the Astronaut table.

In this assignment, you'll just have to create tables with the correct table names, attributes, data types, Primary Keys and Referential Integrity. "Must appear as" means that there's a Referential Integrity requirement. **Be sure not to forget Primary Keys and Referential Integrity when you do Lab1!**

4.2.1 Data types

- For payloadID, astronautID, missionID, contractID, massKg, numLaunchPads, altitudeMeters, durationHours, numTechnicians, and yearsExperience use integer.
- For payloadName, owner, siteCode, launchSiteCode, siteName, country, astronautName, agency, rocketType, paymentMethod, baySlot, and role use character of variable length, with maximum length 40.
- For payloadType, outcome, and priority use character with fixed length 1. We'll say more about the values of these attributes later.
- For dateOfBirth and signedDate use the date type.
- For scheduledLaunch, actualLaunch, and integrationTime use the timestamp type.
- For price and compensation use numeric, with at most 7 digits to the left of the decimal point and 2 digits after it.

You must write a CREATE TABLE statement for each of the 7 tables. Write the statements in the same order that the tables are listed above. **Use the data types, Primary Keys and Referential Integrity described above.** Don't do anything beyond that, even if you think that it's sensible. Save your statements in the file create_lab1.sql.

PostgreSQL maps all SQL identifiers (e.g., table names and attributes) to lowercase. That's okay in your assignments. You won't lose points for Lab1 because Astronaut appears in the database as astronaut. It is possible to specify specific case choices for an identifier by putting that identifier inside double-quote symbols, e.g., as "Astronaut". But then every time you refer to that identifier, you'll have to use the double-quotes. "ASTRONAUT" is not the same identifier as "Astronaut", and neither of those is the same as Astronaut (written without double-quotes), which PostgreSQL maps to astronaut. We will use capitalization for readability, but we won't bother using double-quotes in our own Lectures, Lab Assignments and Exams.

5. Testing

While you're working on your solution, it is a good idea to drop all objects from the schema every time you run the create_lab1.sql script, so you can start fresh. Dropping each object in a schema may be tedious, and sometimes there may be a particular order in which objects must be dropped. (Why?) The following command, which you should put at the top of your create_lab1.sql script, will drop your Lab1 schema (and all the objects within it), and then create the (empty) schema again:

```
DROP SCHEMA Lab1 CASCADE;
CREATE SCHEMA Lab1;
```

The first statement will result in an error if the Lab1 Schema doesn't exist, but execution of your script will continue after that.

Before you submit your Lab1 solution, login to your database via psql and execute your create_lab1.sql script. As you'll learn in Lab Sections, the command to execute a script is: \i <filename> Since psql runs inside the Docker container, it can only see the files in your CSE180-lab folder, which is mounted at /CSE180-lab inside the container. Keep your .sql files in that folder, and in psql either run **\cd /CSE180-lab** once and then **\i create_lab1.sql**, or give the full path: **\i /CSE180-lab/create_lab1.sql**. The same applies to load_lab1.sql below. Verify that every table has been created by using the command: \d When you execute \d, the tables may be displayed in any order, not necessarily in the order in which you created them.

To verify that the attributes of each table are in the correct order, and that each attribute is assigned its correct data type use the following command: \d <table>.

We've supplied some load data that you can use to test your solution in the file load_lab1.sql. After you've created your tables, using the command : \i create_lab1.sql, you can load that data in psql by executing the command: \i load_lab1.sql. (Why will loading the data twice always result in errors?) If your solution fails on the load data, then it's likely that your solution has an error. But although testing can demonstrate that a program is buggy, testing cannot prove that a program is correct.

6. Submitting

1. Save your script as create_lab1.sql You may add informative comments to your scripts if you want. Put any other information for the Graders in a separate README file that you may submit. If you used an AI tool, the README is where you disclose it (see Policies).

2. Zip the file(s) to a single file with name Lab1_XXXXXXX.zip where XXXXXXX is your 7-digit student ID. For example, if a student's ID is 1234567, then the file that this student submits for Lab1 should be named Lab1_1234567.zip

   If you have a README file (which is not required), you can use the Unix command:

   zip Lab1_1234567 create_lab1.sql README

   If you don't have a README file, to create the zip file you can use the Unix command:

   zip Lab1_1234567 create_lab1.sql

   (Of course, you should use **your own student ID**, not 1234567.) Submit a zip file, even if you only have one file.

   Upload the zip file and, in the same Canvas submission, also upload each file individually (create_lab1.sql, and README if you have one). This lets us grade you even if the zip is broken. A submission with only a zip that turns out to be empty or wrong cannot be fixed after the solutions are posted.

Submit the zip file and the individual files on Canvas under Assignment Lab1. Please be sure that you have access to Canvas. Registered students should automatically have access; students who are not registered will not be able to submit solutions. You can resubmit as many times as you like. (Canvas will give the new file a slightly different name, but that's okay.)

The TAs will help you set up PostgreSQL in a docker container and show you how to move and zip the necessary files in order to submit them during Lab Sections. Attend your Lab Section to ensure that you know how to handle this correctly!

Lab1 is due by 11:59pm on Friday, October 9. **See Policies at the top of this document for the late window.** Always check to make sure that your submission is on Canvas, and that you've submitted the correct file.